

Planificación Detallada de la Versión Alpha v2.0: Órganos

1. Fundamentos de la Fase Alpha v2.0

Habiendo completado la Fase Alpha v1 ("Articulaciones"), donde conectamos la infraestructura básica con las primeras herramientas de cálculo, la **Fase Alpha v2.0 ("Órganos")** se enfoca en implementar las funcionalidades centrales y más complejas que definen la propuesta de valor de ChemsTools. Esta etapa representa el corazón de la aplicación, donde pasamos de herramientas de consulta a módulos interactivos de análisis químico.

El software en esta fase seguirá siendo para pruebas internas. El objetivo es la implementación completa de la lógica de negocio principal y la validación de las integraciones con librerías químicas especializadas como RDKit y SymPy.

2. Objetivos Generales de la Fase Alpha v2.0

- **Implementar Módulos Químicos Avanzados:** Desarrollar el Generador de Estructuras de Lewis y un Simulador de Reacciones Químicas básico.
- **Integrar Librerías Científicas Clave:** Incorporar RDKit para la manipulación y análisis de estructuras moleculares y SymPy para el balanceo de ecuaciones simbólicas en el backend.
- **Desarrollar Interfaces de Usuario Especializadas:** Crear los componentes de frontend necesarios para interactuar con los nuevos módulos, incluyendo la visualización de estructuras 2D con una librería como ChemDoodle Web Components.
- **Ampliar la Cobertura de Pruebas:** Asegurar que toda la nueva lógica de negocio crítica esté cubierta por pruebas unitarias y de integración robustas.
- **Preparar el Terreno para la Fase Beta:** Alcanzar un estado "feature-complete" para las funcionalidades básicas, dejando la aplicación lista para las fases de optimización y pruebas con usuarios externos.

3. Estructura de Versiones Alpha v2

Dividiremos la fase en sub-versiones para un desarrollo incremental y controlado.

Alpha 2.0.0: Infraestructura y Librerías Científicas

- **Objetivo:** Preparar la arquitectura del backend para las nuevas funcionalidades, integrando las librerías necesarias.
- **Backend:**
 - **Integración de Librerías:** Añadir rdkit-pypi y sympy al archivo backend/requirements.txt y reconstruir el contenedor para asegurar que se instalen correctamente.
 - **Nuevas Apps de Django:** Crear dos nuevas aplicaciones para organizar la lógica:
 - structures: Alojará los modelos y la lógica relacionados con estructuras moleculares (ej. Lewis).
 - reactions: Contendrá la lógica para el balanceo y simulación de reacciones.
 - Registrar las nuevas apps en chems_tools/settings.py.
- **Frontend:**
 - Investigar y seleccionar la librería de visualización 2D (ej. ChemDoodle Web Components, Kekule.js). Instalar la dependencia seleccionada.
- **DevOps:**
 - Actualizar el pipeline de CI para instalar las nuevas dependencias de Python y ejecutar las pruebas de las nuevas apps.

Alpha 2.1.0: Módulo Generador de Estructuras de Lewis

- **Objetivo:** Implementar la funcionalidad completa para generar y visualizar estructuras de Lewis.
- **Backend:**
 - Dentro de la app structures, desarrollar un nuevo endpoint de API (ej. `/api/structures/lewis-generator/`).
 - Este endpoint aceptará una fórmula molecular simple (ej. "H2O", "CH4") y utilizará RDKit para determinar la conectividad y los electrones de valencia, devolviendo una representación de la estructura de Lewis (ej. en formato MOL o un JSON personalizado).
- **Frontend:**
 - Crear un nuevo componente `LewisStructureGenerator.tsx`.
 - Diseñar un formulario donde el usuario pueda introducir una fórmula.
 - Al recibir la respuesta de la API, utilizar la librería de visualización (ej. ChemDoodle) para renderizar la estructura 2D en un canvas.
- **Pruebas:**
 - Añadir pruebas unitarias para el servicio de generación de estructuras en el backend, validando casos comunes (agua, metano, amoníaco, etc.).

Alpha 2.2.0: Módulo Simulador de Reacciones

- **Objetivo:** Implementar el balanceo de ecuaciones y la predicción básica de productos.
- **Backend:**
 - Dentro de la app reactions, desarrollar un endpoint de API (ej. `/api/reactions/balance-equation/`).
 - Este endpoint aceptará una ecuación química no balanceada como string (ej. `"H2 + O2 -> H2O"`).
 - Utilizar SymPy para analizar la ecuación, balancearla y devolver la ecuación balanceada junto con los coeficientes.
- **Frontend:**
 - Crear un nuevo componente `ReactionSimulator.tsx`.
 - Diseñar una interfaz donde el usuario pueda escribir los reactivos y productos de una ecuación.
 - Al hacer clic en "Balancear", enviar la petición a la API y mostrar la ecuación balanceada en la interfaz.
- **Pruebas:**
 - Añadir pruebas unitarias para el servicio de balanceo de ecuaciones, cubriendo reacciones simples de síntesis, descomposición y combustión.

Alpha 2.3.0: Refinamiento, Integración y Documentación

- **Objetivo:** Estabilizar las nuevas funcionalidades, integrar notificaciones y actualizar la documentación.
- **Backend:**
 - Refactorizar el código de las apps `structures` y `reactions` basándose en los hallazgos de las pruebas.
 - Añadir docstrings detallados a las nuevas vistas para mejorar la documentación autogenerada de la API.
- **Frontend:**
 - Integrar el sistema de notificaciones global en los nuevos componentes para un feedback consistente (éxito/error).
 - Realizar una revisión de la UI/UX de las nuevas herramientas, ajustando el diseño para una mejor usabilidad.
- **Documentación:**
 - Verificar que la documentación de Swagger/Redoc para los nuevos endpoints es clara y completa.
 - Añadir las nuevas secciones correspondientes a la `USER_GUIDE.md`.

4. Gestión de Entregables por Versión (Alpha v2)

- **Documentación Técnica:**
 - Documentación de API para /api/structures/lewis-generator/ y /api/reactions/balance-equation/.
 - Diagramas de arquitectura actualizados que reflejen las nuevas apps structures y reactions.
 - USER_GUIDE.md actualizada con guías para las nuevas funcionalidades.
- **Entregables de Código:**
 - **Backend:** Módulos de Django structures y reactions con sus servicios y endpoints funcionales.
 - **Frontend:** Componentes de React LewisStructureGenerator.tsx y ReactionSimulator.tsx integrados en la aplicación.
- **Pruebas:**
 - Suite de pruebas unitarias con una cobertura superior al 80% para el nuevo código del backend.
 - Pruebas de componentes para los nuevos flujos de usuario en el frontend.

5. Métricas de Evaluación para la Fase Alpha v2

- **Calidad del Código:**
 - **Cobertura de Pruebas:** Mantener o superar el 80% en los módulos structures y reactions.
 - **Deuda Técnica:** Cero problemas críticos o bloqueantes reportados por los linters.
- **Progreso del Proyecto:**
 - **Cumplimiento de Funcionalidades:** El 100% de las características definidas para Alpha v2 deben estar implementadas y funcionales en el entorno de desarrollo.
- **Rendimiento y Estabilidad:**
 - **Tiempo de Respuesta de la API:** Los nuevos endpoints de cálculo deben responder consistentemente en menos de 500ms.
 - **Estabilidad de la Build:** La rama develop debe mantener una tasa de éxito en los builds de CI superior al 95%.
 - **Bugs Críticos:** No debe haber bugs críticos sin resolver al final de la fase.